

Списки (Lists), Подпрограммы: Функции vs Процедуры

Структура урока: 30 минут — Теория и разбор ЕНТ-ловушек | 60 минут — Практический тренажер

1. Функции и Процедуры в Python (15 минут)

В Python подпрограммы создаются с помощью ключевого слова `def`. На ЕНТ часто проверяется понимание разницы между функцией и процедурой.

Функция (Function)

Обязательно возвращает значение с помощью ключевого слова `return`. Результат можно присвоить переменной или передать в другое выражение.

```
def add(a, b):  
    return a + b  
  
res = add(3, 5) # res = 8
```

Процедура (Procedure)

Выполняет последовательность действий (вывод, запись), но **не возвращает** явного значения (возвращает `None`).

```
def show_add(a, b):  
    print(a + b)  
  
res = show_add(3, 5) # res = None!
```

⚠ Главная ловушка ЕНТ!

Если функция не содержит оператор `return`, то при попытке присвоить ее результат переменной (например, `x = func()`), переменная `x` получит значение `None`. В тестах часто дают код с `print()` внутри `def` без `return` и спрашивают значение вызова!

2. Списки (Lists) в Python (15 минут)

Список — это упорядоченная изменяемая коллекция элементов. Индексация начинается с 0.

Метод / Операция	Описание	Пример
<code>append(x)</code>	Добавляет элемент <code>x</code> в конец списка	<code>a = [1]; a.append(2) → [1, 2]</code>
<code>insert(i, x)</code>	Вставляет элемент <code>x</code> на позицию с индексом <code>i</code>	<code>a = [1, 3]; a.insert(1, 2) → [1, 2, 3]</code>
<code>pop(i)</code>	Удаляет и возвращает элемент по индексу <code>i</code> (по умолч. последний)	<code>a = [10, 20]; x = a.pop(0) → x=10, a=[20]</code>
<code>remove(x)</code>	Удаляет первый найденный элемент со значением <code>x</code>	<code>a = [5, 9, 5]; a.remove(5) → [9, 5]</code>
Срезы (Slices)	<code>list[start:stop:step]</code> (stop не включается)	<code>a = [0,1,2,3,4]; a[1:4] → [1, 2, 3]</code>

Критически важные встроенные функции для ЕНТ:

`len(lst)` — длина списка | `sum(lst)` — сумма элементов | `max(lst)` / `min(lst)` — макс/мин элемент | `sorted(lst)` — возвращает новый отсортированный список.

Часть 1: Тестовые задания ЕНТ (30 минут)

1 балл

```
def calc(a, b):
    res = a * b + 2
    print(res)

a = calc(3, 4)
print(a)
```

- 1 балл

1 балл

```
def modify(lst):
    lst.append(100)
    lst = [1, 2, 3]
    lst.append(200)

my_list = [10, 20]
modify(my_list)
print(my_list)
```

- 1 балл

```
arr = [4, 1, 3, 2]
arr.sort()
arr.pop(1)
arr.insert(2, 5)
print(arr)
```

- 2 балла

```
data = [2, 5, 8, 11, 14]
for i in range(len(data)):
```

ЗАДАЧИ НА НАПИСАНИЕ КОДА

Часть 2: Практическое программирование (30 минут)

Задание 6. Написание функции (ЕНТ контекст)

3 балла

Условие: Напишите функцию `filter_even(numbers)`, которая принимает список чисел и возвращает новый список, содержащий только четные числа из исходного списка, увеличенные в 3 раза.

Пример входных данных: `[1, 2, 3, 4, 5, 6]` → Результат: `[6, 12, 18]`

Задание 7. Написание процедуры

3 балла

Условие: Напишите процедуру `print_stats(numbers)`, которая принимает список чисел и выводит на экран в две строки:

- 1-я строка: Среднее арифметическое элементов списка (округленное до 2 знаков).
- 2-я строка: Разницу между максимальным и минимальным элементом.

Ключи и разбор решений (Для преподавателя / Самопроверка)

№ Задания	Правильный ответ	Краткое объяснение
Задание 1	A	Функция <code>calc</code> выполняет <code>print(14)</code> . Так как нет <code>return</code> , вызов <code>calc(3, 4)</code> возвращает <code>None</code> , который затем выводится вторым <code>print</code> .
Задание 2	A	Срез <code>[1:6:2]</code> берет элементы с индексами 1, 3, 5. Это элементы <code>20, 40, 60</code> .
Задание 3	A	<code>lst.append(100)</code> изменяет исходный список. Переприсваивание <code>lst = [1, 2, 3]</code> внутри функции создает новую локальную ссылку, не влияя на <code>my_list</code> .
Задание 4	A	Сортировка: <code>[1, 2, 3, 4].pop(1)</code> удаляет '2' → <code>[1, 3, 4]</code> . <code>insert(2, 5)</code> вставляет 5 на индекс 2 → <code>[1, 3, 5, 4]</code> .
Задание 5	D	Четные элементы (2, 8, 14) делятся на 2: становятся (1, 4, 7). Нечетные (5, 11) остаются. Новый список: <code>[1, 5, 4, 11, 7]</code> . Сумма = 1 + 5 + 4 + 11 + 7 = 23.

Образец решения Задания 6:

```
def filter_even(numbers):
    res = []
    for x in numbers:
        if x % 2 == 0:
            res.append(x * 3)
    return res
```

